CS-305-MJ-P: Operating System-I

Slip-wise Solutions (T.Y.B.Sc. CS, SPPU, Sem V) — Q1 and Q2 of Selected Slips

Selected slips: 1, 3, 4, 5, 9, 10, 11, 12, 13, 16, 20, 23, 25, 27, 28. Both questions (Q1 and Q2, 15 marks each) are solved for every slip. When the same program repeats in another slip it is written out again in full, so each slip stands on its own. Every program is plain C with no comments, was compiled with gcc and run on the data shown; the sample output is the real output. The shell programs follow the style of your sample. Compile: `gcc file.c -o out` Run: `./out`

SLIP 1

Q1 (15 marks)

Question: Write a C menu driven program: a) Accept Available b) Display Allocation, Max c) Display the contents of need matrix d) Display Available (5 processes, 3 resource types A B C, data as given in the question).

Program

```
#include <stdio.h>

int alloc[5][3] = {{2, 3, 2}, {4, 0, 0}, {5, 0, 4}, {4, 3, 3}, {2, 2, 4}};
int max[5][3]   = {{9, 7, 5}, {5, 2, 2}, {1, 0, 4}, {4, 4, 4}, {6, 5, 5}};
int avail[3];
int need[5][3];

int main() {
    char ch;
    int i, j;

    do {
        printf("\na) Accept Available\n");
        printf("b) Display Allocation, Max\n");
        printf("c) Display contents of Need matrix\n");
        printf("d) Display Available\n");
        printf("e) Exit\n");
        printf("Enter your choice: ");
        scanf(" %c", &ch);

        switch (ch) {
        case 'a':
            printf("Enter Available resources (A B C): ");
            for (j = 0; j < 3; j++)
                scanf("%d", &avail[j]);
            break;
        case 'b':
            printf("\nProcess\tAllocation\tMax\n");
            printf("\tA B C\t\tA B C\n");
            for (i = 0; i < 5; i++) {
                printf("P%d\t", i);
                for (j = 0; j < 3; j++)
                    printf("%d ", alloc[i][j]);
                printf("\t\t");
                for (j = 0; j < 3; j++)
                    printf("%d ", max[i][j]);
                printf("\n");
            }
            break;
        case 'c':
            printf("\nNeed Matrix (Max - Allocation)\n");
            printf("\tA B C\n");
            for (i = 0; i < 5; i++) {
                printf("P%d\t", i);
                for (j = 0; j < 3; j++) {
                    need[i][j] = max[i][j] - alloc[i][j];
                    printf("%d ", need[i][j]);
                }
                printf("\n");
            }
            break;
        case 'd':
            printf("Available: A=%d B=%d C=%d\n", avail[0], avail[1], avail[2]);
            break;
```

```
        case 'e':
            printf("Exiting...\n");
            break;
        default:
            printf("Invalid choice\n");
        }
    } while (ch != 'e');
    return 0;
}
```

Sample Input / Output

```
a) Accept Available
b) Display Allocation, Max
c) Display contents of Need matrix
d) Display Available
e) Exit
Enter your choice: a
Enter Available resources (A B C): 3 3 2

a) Accept Available
b) Display Allocation, Max
c) Display contents of Need matrix
d) Display Available
e) Exit
Enter your choice: b

Process Allocation      Max
        A B C           A B C
P0      2 3 2           9 7 5
P1      4 0 0           5 2 2
P2      5 0 4           1 0 4
P3      4 3 3           4 4 4
P4      2 2 4           6 5 5

a) Accept Available
b) Display Allocation, Max
c) Display contents of Need matrix
d) Display Available
e) Exit
Enter your choice: c

Need Matrix (Max - Allocation)
        A B C
P0      7 4 3
P1      1 2 2
P2      -4 0 0
P3      0 1 1
P4      4 3 1

a) Accept Available
b) Display Allocation, Max
c) Display contents of Need matrix
d) Display Available
e) Exit
Enter your choice: d
Available: A=3 B=3 C=2

a) Accept Available
b) Display Allocation, Max
c) Display contents of Need matrix
d) Display Available
e) Exit
Enter your choice: e
Exiting...
```

Note: in the question's data some Max values are smaller than Allocation, so those need values come out negative. That is how the given data works out.

Q2 (15 marks)

Question: Write a C program that behaves like a shell which displays a command prompt. It accepts the command, tokenizes the command line and executes it by creating a child process. Also implement the additional command count: count c filename (number of characters), count w filename (number of words), count l filename (number of lines).

Program

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

void make_toks(char *s, char *tok[])
{
 int i = 0;
 char *p;
 p = strtok(s, " ");
 while (p != NULL)
 {
  tok[i++] = p;
  p = strtok(NULL, " ");
 }
 tok[i] = NULL;
}

void count(char *fn, char op)
{
 int fh, cc = 0, wc = 0, lc = 0;
 char c;
 fh = open(fn, O_RDONLY);
 if (fh == -1)
 {
  printf("File %s not found.\n", fn);
  return;
 }
 while (read(fh, &c, 1) > 0)
 {
  if (c == ' ')
   wc++;
  else if (c == '\n')
  {
   wc++;
   lc++;
  }
  cc++;
 }
 close(fh);
 switch (op)
 {
 case 'c':
  printf("No.of characters:%d\n", cc);
  break;
 case 'w':
  printf("No.of words:%d\n", wc);
  break;
 case 'l':
  printf("No.of lines:%d\n", lc);
  break;
 }
}

int main()
{
 char buff[80], *args[10];
 int pid;
 while (1)
 {
  printf("myshell$ ");
  fflush(stdout);
  fgets(buff, 80, stdin);
  buff[strlen(buff) - 1] = '\0';
  make_toks(buff, args);
```

```
  if (args[0] == NULL)
   continue;
  if (strcmp(args[0], "quit") == 0)
   break;
  else if (strcmp(args[0], "count") == 0)
  {
   if (args[1] == NULL || args[2] == NULL)
    printf("Usage: count c|w|l filename\n");
   else
    count(args[2], args[1][0]);
  }
  else
  {
   pid = fork();
   if (pid > 0)
    wait(NULL);
   else
   {
    if (execvp(args[0], args) == -1)
     printf("Bad command\n");
    exit(1);
   }
  }
 }
 return 0;
}
```

Sample Input / Output

```
myshell$ count c test.txt
No.of characters:39
myshell$ count w test.txt
No.of words:8
myshell$ count l test.txt
No.of lines:3
myshell$ ls test.txt
test.txt
myshell$ xyz
Bad command
myshell$ quit
```

Shell style follows your sample (myshell$ prompt, make_toks, count, fork/wait/execvp). Create a small test.txt before running. Words are counted by spaces and newlines, as in the sample.

SLIP 3

Q1 (15 marks)

Question: Write a C program to simulate Banker's algorithm for deadlock avoidance (5 processes P0-P4, 4 resource types A B C D, Allocation, Max and Available as given). a) Calculate and display the need matrix. b) Is the system in a safe state? If yes, display the safe sequence.

Program

```
#include <stdio.h>

int main() {
    int alloc[5][4] = {{0, 0, 1, 2}, {1, 0, 0, 0}, {1, 3, 5, 4}, {0, 6, 3, 2}, {0, 0, 1, 4}};
    int max[5][4]   = {{0, 0, 1, 2}, {1, 7, 5, 0}, {2, 3, 5, 6}, {0, 6, 5, 2}, {0, 6, 5, 6}};
    int avail[4]    = {1, 5, 2, 0};
    int need[5][4], work[4], finish[5] = {0}, seq[5];
    int i, j, count = 0, found, ok;

    printf("Need Matrix (Max - Allocation)\n");
    printf("\tA B C D\n");
    for (i = 0; i < 5; i++) {
        printf("P%d\t", i);
        for (j = 0; j < 4; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }

    for (j = 0; j < 4; j++)
        work[j] = avail[j];

    while (count < 5) {
        found = 0;
        for (i = 0; i < 5; i++) {
            if (!finish[i]) {
                ok = 1;
                for (j = 0; j < 4; j++)
                    if (need[i][j] > work[j]) {
                        ok = 0;
                        break;
                    }
                if (ok) {
                    for (j = 0; j < 4; j++)
                        work[j] += alloc[i][j];
                    seq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if (!found)
            break;
    }

    if (count == 5) {
        printf("\nThe system is in SAFE state.\nSafe sequence: ");
        for (i = 0; i < 5; i++) {
            printf("P%d", seq[i]);
            if (i < 4)
                printf(" -> ");
        }
        printf("\n");
    } else {
        printf("\nThe system is NOT in safe state.\n");
    }
    return 0;
}
```

Sample Input / Output

```
Need Matrix (Max - Allocation)
        A B C D
P0      0 0 0 0
P1      0 7 5 0
```

```
P2      1 0 0 2
P3      0 0 2 0
P4      0 6 4 2

The system is in SAFE state.
Safe sequence: P0 -> P2 -> P3 -> P4 -> P1
```

Q2 (15 marks)

Question: Write a simulation program for disk scheduling using the SSTF algorithm. Requests: 98, 183, 37, 122, 14, 124, 65. Start head position: 53.

Program

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, blocks, head, i, j, total = 0, served = 0;
    int req[50], done[50] = {0};

    printf("Enter total number of disk blocks: ");
    scanf("%d", &blocks);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &req[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);

    printf("\nOrder of service: %d", head);
    while (served < n) {
        int min = 100000, idx = -1;
        for (j = 0; j < n; j++) {
            if (!done[j] && abs(req[j] - head) < min) {
                min = abs(req[j] - head);
                idx = j;
            }
        }
        done[idx] = 1;
        total += min;
        head = req[idx];
        printf(" -> %d", head);
        served++;
    }
    printf("\nTotal head movements = %d\n", total);
    return 0;
}
```

Sample Input / Output

```
Enter total number of disk blocks: 200
Enter number of requests: 7
Enter request string: 98 183 37 122 14 124 65
Enter current head position: 53

Order of service: 53 -> 65 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total head movements = 232
```

SLIP 4

Q1 (15 marks)

Question: Implement the menu driven Banker's algorithm for accepting Allocation, Max from the user: a) Accept Available b) Display Allocation, Max c) Find Need and display it d) Display Available. System with 3 resource types A, B, C and 5 processes as in the given snapshot.

Program

```
#include <stdio.h>

int alloc[10][10], max[10][10], need[10][10], avail[10];
int n, m;

int main() {
    char ch;
    int i, j;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    printf("Enter Allocation matrix (%d x %d):\n", n, m);
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter Max matrix (%d x %d):\n", n, m);
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    do {
        printf("\na) Accept Available\n");
        printf("b) Display Allocation, Max\n");
        printf("c) Find Need and display it\n");
        printf("d) Display Available\n");
        printf("e) Exit\n");
        printf("Enter your choice: ");
        scanf(" %c", &ch);

        switch (ch) {
        case 'a':
            printf("Enter Available resources: ");
            for (j = 0; j < m; j++)
                scanf("%d", &avail[j]);
            break;
        case 'b':
            printf("\nProcess\tAllocation\tMax\n\t");
            for (j = 0; j < m; j++)
                printf("%c ", 'A' + j);
            printf("\t\t");
            for (j = 0; j < m; j++)
                printf("%c ", 'A' + j);
            printf("\n");
            for (i = 0; i < n; i++) {
                printf("P%d\t", i);
                for (j = 0; j < m; j++)
                    printf("%d ", alloc[i][j]);
                printf("\t\t");
                for (j = 0; j < m; j++)
                    printf("%d ", max[i][j]);
                printf("\n");
            }
            break;
        case 'c':
            printf("\nNeed Matrix (Max - Allocation)\n\t");
            for (j = 0; j < m; j++)
                printf("%c ", 'A' + j);
            printf("\n");
            for (i = 0; i < n; i++) {
                printf("P%d\t", i);
                for (j = 0; j < m; j++) {
                    need[i][j] = max[i][j] - alloc[i][j];
                    printf("%d ", need[i][j]);
```

```
                }
                printf("\n");
            }
            break;
        case 'd':
            printf("Available: ");
            for (j = 0; j < m; j++)
                printf("%c=%d ", 'A' + j, avail[j]);
            printf("\n");
            break;
        case 'e':
            printf("Exiting...\n");
            break;
        default:
            printf("Invalid choice\n");
        }
    } while (ch != 'e');
    return 0;
}
```

Sample Input / Output

```
Enter number of processes: 5
Enter number of resource types: 3
Enter Allocation matrix (5 x 3):
0 1 0
4 0 0
5 0 4
4 3 3
2 2 4
Enter Max matrix (5 x 3):
0 0 0
5 2 2
1 0 4
4 4 4
6 5 5

a) Accept Available
b) Display Allocation, Max
c) Find Need and display it
d) Display Available
e) Exit
Enter your choice: a
Enter Available resources: 7 2 6

a) Accept Available
b) Display Allocation, Max
c) Find Need and display it
d) Display Available
e) Exit
Enter your choice: b

Process Allocation      Max
        A B C           A B C
P0      0 1 0           0 0 0
P1      4 0 0           5 2 2
P2      5 0 4           1 0 4
P3      4 3 3           4 4 4
P4      2 2 4           6 5 5

a) Accept Available
b) Display Allocation, Max
c) Find Need and display it
d) Display Available
e) Exit
Enter your choice: c

Need Matrix (Max - Allocation)
        A B C
P0      0 -1 0
P1      1 2 2
P2      -4 0 0
P3      0 1 1
P4      4 3 1

a) Accept Available
b) Display Allocation, Max
c) Find Need and display it
d) Display Available
e) Exit
```

```
Enter your choice: d
Available: A=7 B=2 C=6

a) Accept Available
b) Display Allocation, Max
c) Find Need and display it
d) Display Available
e) Exit
Enter your choice: e
Exiting...
```

Note: in the question's data some Max values are smaller than Allocation, so those need values come out negative. That is how the given data works out.

Q2 (15 marks)

Question: Write a simulation program for disk scheduling using the SCAN algorithm. Requests: 82, 170, 43, 140, 24, 16, 190, 65. Starting head position 50, direction Left.

Program

```
#include <stdio.h>
#include <stdlib.h>

int cur, total = 0;

void go(int x) {
    printf(" -> %d", x);
    total += abs(x - cur);
    cur = x;
}

int main() {
    int req[50], n, blocks, head, dir, i, j, t, pos;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &blocks);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &req[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): ");
    scanf("%d", &dir);

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (req[j] > req[j + 1]) {
                t = req[j];
                req[j] = req[j + 1];
                req[j + 1] = t;
            }

    for (pos = 0; pos < n && req[pos] < head; pos++);

    cur = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0) {
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
        go(0);
        for (i = pos; i < n; i++)
            go(req[i]);
    } else {
        for (i = pos; i < n; i++)
            go(req[i]);
        go(blocks - 1);
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
    }
    printf("\nTotal head movements = %d\n", total);
    return 0;
}
```

Sample Input / Output

```
Enter total number of disk blocks: 200
Enter number of requests: 8
Enter request string: 82 170 43 140 24 16 190 65
Enter current head position: 50
Enter direction (0 = Left, 1 = Right): 0

Order of service: 50 -> 43 -> 24 -> 16 -> 0 -> 65 -> 82 -> 140 -> 170 -> 190
Total head movements = 240
```

SLIP 5

Q1 (15 marks)

Question: Write the simulation program for demand paging and show the page scheduling and total number of page faults according to the FIFO page replacement algorithm. Assume the memory of n frames. Reference string: 3,4,5,6,3,4,7,3,4,5,6,7,2,4,6.

Program

```
#include <stdio.h>

int main() {
    int ref[50], frames[10], n, f, i, j, faults = 0, hit, next = 0;

    printf("Enter number of frames: ");
    scanf("%d", &f);
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    for (i = 0; i < f; i++)
        frames[i] = -1;

    printf("\nRef\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        hit = 0;
        for (j = 0; j < f; j++)
            if (frames[j] == ref[i]) {
                hit = 1;
                break;
            }
        if (!hit) {
            frames[next] = ref[i];
            next = (next + 1) % f;
            faults++;
        }
        printf("%d\t", ref[i]);
        for (j = 0; j < f; j++) {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
        printf("\t\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nTotal page faults = %d\n", faults);
    return 0;
}
```

Sample Input / Output

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 3 4 5 6 3 4 7 3 4 5 6 7 2 4 6

Ref     Frames          Status
3       3 - -           Fault
4       3 4 -           Fault
5       3 4 5           Fault
6       6 4 5           Fault
3       6 3 5           Fault
4       6 3 4           Fault
7       7 3 4           Fault
3       7 3 4           Hit
4       7 3 4           Hit
5       7 5 4           Fault
6       7 5 6           Fault
7       7 5 6           Hit
2       2 5 6           Fault
4       2 4 6           Fault
6       2 4 6           Hit

Total page faults = 11
```

Q2 (15 marks)

Question: Consider a system with n processes and m resource types. Accept the number of instances of every resource type. For each process accept the allocation and maximum requirement matrices. Display the contents of the need matrix and check whether the given request of a process can be granted immediately or not (use the resource request algorithm).

Program

```
#include <stdio.h>

int n, m;
int alloc[10][10], max[10][10], need[10][10], avail[10], total[10];

int is_safe() {
    int work[10], finish[10] = {0}, i, j, count = 0, found, ok;

    for (j = 0; j < m; j++)
        work[j] = avail[j];
    while (count < n) {
        found = 0;
        for (i = 0; i < n; i++) {
            if (!finish[i]) {
                ok = 1;
                for (j = 0; j < m; j++)
                    if (need[i][j] > work[j]) {
                        ok = 0;
                        break;
                    }
                if (ok) {
                    for (j = 0; j < m; j++)
                        work[j] += alloc[i][j];
                    finish[i] = 1;
                    count++;
                    found = 1;
                }
            }
        }
        if (!found)
            return 0;
    }
    return 1;
}

int main() {
    int i, j, p, req[10];

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    printf("Enter total instances of each resource type: ");
    for (j = 0; j < m; j++)
        scanf("%d", &total[j]);
    printf("Enter Allocation matrix (%d x %d):\n", n, m);
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter Max matrix (%d x %d):\n", n, m);
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    for (j = 0; j < m; j++) {
        avail[j] = total[j];
        for (i = 0; i < n; i++)
            avail[j] -= alloc[i][j];
    }

    printf("\nNeed Matrix\n\t");
    for (j = 0; j < m; j++)
        printf("%c ", 'A' + j);
    printf("\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t", i);
        for (j = 0; j < m; j++) {
```

```
            need[i][j] = max[i][j] - alloc[i][j];
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }
    printf("Available: ");
    for (j = 0; j < m; j++)
        printf("%d ", avail[j]);
    printf("\n");

    printf("\nEnter process number making the request: ");
    scanf("%d", &p);
    printf("Enter the request: ");
    for (j = 0; j < m; j++)
        scanf("%d", &req[j]);

    for (j = 0; j < m; j++)
        if (req[j] > need[p][j]) {
            printf("Error: process has exceeded its maximum claim.\n");
            return 0;
        }
    for (j = 0; j < m; j++)
        if (req[j] > avail[j]) {
            printf("Process P%d must wait: resources are not available.\n", p);
            return 0;
        }

    for (j = 0; j < m; j++) {
        avail[j] -= req[j];
        alloc[p][j] += req[j];
        need[p][j] -= req[j];
    }
    if (is_safe()) {
        printf("Request can be granted immediately (system stays in safe state).\n");
    } else {
        printf("Request cannot be granted immediately (system would be unsafe).\n");
    }
    return 0;
}
```

Sample Input / Output

```
Enter number of processes: 5
Enter number of resource types: 3
Enter total instances of each resource type: 10 5 7
Enter Allocation matrix (5 x 3):
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max matrix (5 x 3):
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Need Matrix
        A B C
P0      7 4 3
P1      1 2 2
P2      6 0 0
P3      0 1 1
P4      4 3 1
Available: 3 3 2

Enter process number making the request: 1
Enter the request: 1 0 2
Request can be granted immediately (system stays in safe state).
```

Sample data is the standard 5-process, 3-resource example; any other data can be typed in.

SLIP 9

Q1 (15 marks)

Question: Write the simulation program for demand paging and show the page scheduling and total number of page faults according to the Optimal page replacement algorithm. Assume the memory of n frames. Reference string: 8,5,7,8,5,7,2,3,7,3,5,9,4,6,2.

Program

```
#include <stdio.h>

int main() {
    int ref[50], frames[10], n, f, i, j, k, faults = 0, hit, filled = 0;
    int farthest, victim;

    printf("Enter number of frames: ");
    scanf("%d", &f);
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    for (i = 0; i < f; i++)
        frames[i] = -1;

    printf("\nRef\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        hit = 0;
        for (j = 0; j < f; j++)
            if (frames[j] == ref[i]) {
                hit = 1;
                break;
            }
        if (!hit) {
            faults++;
            if (filled < f) {
                frames[filled++] = ref[i];
            } else {
                farthest = -1;
                victim = 0;
                for (j = 0; j < f; j++) {
                    for (k = i + 1; k < n; k++)
                        if (frames[j] == ref[k])
                            break;
                    if (k > farthest) {
                        farthest = k;
                        victim = j;
                    }
                }
                frames[victim] = ref[i];
            }
        }
        printf("%d\t", ref[i]);
        for (j = 0; j < f; j++) {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
        printf("\t\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nTotal page faults = %d\n", faults);
    return 0;
}
```

Sample Input / Output

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 8 5 7 8 5 7 2 3 7 3 5 9 4 6 2

Ref     Frames          Status
8       8 - -           Fault
5       8 5 -           Fault
```

```
7       8 5 7           Fault
8       8 5 7           Hit
5       8 5 7           Hit
7       8 5 7           Hit
2       2 5 7           Fault
3       3 5 7           Fault
7       3 5 7           Hit
3       3 5 7           Hit
5       3 5 7           Hit
9       9 5 7           Fault
4       4 5 7           Fault
6       6 5 7           Fault
2       2 5 7           Fault

Total page faults = 9
```

Q2 (15 marks)

Question: Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU burst for n processes are input, the next CPU burst is generated randomly. Output the Gantt chart, turnaround time and waiting time of each process and the average waiting and turnaround time.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void gantt(int gp[], int gs[], int ge[], int g) {
    int i;
    printf("\nGantt Chart:\n");
    for (i = 0; i < g; i++) {
        if (gp[i] == 0)
            printf("| Idle ");
        else
            printf("|  P%d  ", gp[i]);
    }
    printf("|\n");
    for (i = 0; i < g; i++)
        printf("%-7d", gs[i]);
    printf("%d\n", ge[g - 1]);
}

int main() {
    int n, i, t = 0, done = 0, sel, next, tat, wt;
    int at[20], bt[20], ct[20], fin[20] = {0};
    int gp[40], gs[40], ge[40], g = 0;
    float twt = 0, ttat = 0;

    srand(time(NULL));
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter arrival time and burst time for P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    while (done < n) {
        sel = -1;
        for (i = 0; i < n; i++)
            if (!fin[i] && at[i] <= t && (sel == -1 || bt[i] < bt[sel]))
                sel = i;
        if (sel == -1) {
            next = -1;
            for (i = 0; i < n; i++)
                if (!fin[i] && (next == -1 || at[i] < at[next]))
                    next = i;
            gp[g] = 0;
            gs[g] = t;
            ge[g] = at[next];
            g++;
            t = at[next];
            continue;
        }
        gp[g] = sel + 1;
        gs[g] = t;
        t += bt[sel];
```

```
            ge[g] = t;
            g++;
            ct[sel] = t;
            fin[sel] = 1;
            done++;
        }

        gantt(gp, gs, ge, g);
        printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tNext CPU burst\n");
        for (i = 0; i < n; i++) {
            tat = ct[i] - at[i];
            wt = tat - bt[i];
            ttat += tat;
            twt += wt;
            printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt, rand() % 10 + 1);
        }
        printf("\nAverage Turnaround Time = %.2f\n", ttat / n);
        printf("Average Waiting Time    = %.2f\n", twt / n);
        return 0;
    }
```

Sample Input / Output

```
Enter number of processes: 4
Enter arrival time and burst time for P1: 0 7
Enter arrival time and burst time for P2: 2 4
Enter arrival time and burst time for P3: 4 1
Enter arrival time and burst time for P4: 5 4

Gantt Chart:
|  P1  |  P3  |  P2  |  P4  |
0      7      8      12     16

Process AT      BT      CT      TAT     WT      Next CPU burst
P1      0       7       7       7       0       7
P2      2       4       12      10      6       5
P3      4       1       8       4       3       8
P4      5       4       16      11      7       4

Average Turnaround Time = 8.00
Average Waiting Time    = 4.00
```

The question gives no data, so sample data is used. The Next CPU burst column is generated randomly with rand(), as the question asks, so it changes on every run.

SLIP 10

Q1 (15 marks)

Question: Write the program to simulate Round Robin (RR) scheduling. The arrival time and first CPU burst for n processes and the time quantum are input, the next CPU burst is generated randomly. Output the Gantt chart, turnaround time and waiting time of each process and the average waiting and turnaround time.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void gantt(int gp[], int gs[], int ge[], int g) {
    int i;
    printf("\nGantt Chart:\n");
    for (i = 0; i < g; i++) {
        if (gp[i] == 0)
            printf("| Idle ");
        else
            printf("|  P%d  ", gp[i]);
    }
    printf("|\n");
    for (i = 0; i < g; i++)
        printf("%-7d", gs[i]);
    printf("%d\n", ge[g - 1]);
}

int main() {
    int n, i, j, p, run, tq, t = 0, done = 0, next = 0, tat, wt, tmp;
    int at[20], bt[20], rem[20], ct[20], ord[20], q[500], qh = 0, qt = 0;
    int gp[200], gs[200], ge[200], g = 0;
    float twt = 0, ttat = 0;

    srand(time(NULL));
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter arrival time and burst time for P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        rem[i] = bt[i];
        ord[i] = i;
    }
    printf("Enter time quantum: ");
    scanf("%d", &tq);

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (at[ord[j]] > at[ord[j + 1]]) {
                tmp = ord[j];
                ord[j] = ord[j + 1];
                ord[j + 1] = tmp;
            }

    while (done < n) {
        while (next < n && at[ord[next]] <= t)
            q[qt++] = ord[next++];
        if (qh == qt) {
            gp[g] = 0;
            gs[g] = t;
            ge[g] = at[ord[next]];
            g++;
            t = at[ord[next]];
            continue;
        }
        p = q[qh++];
        run = rem[p] < tq ? rem[p] : tq;
        gp[g] = p + 1;
        gs[g] = t;
        t += run;
        ge[g] = t;
        g++;
        rem[p] -= run;
        while (next < n && at[ord[next]] <= t)
```

```
            q[qt++] = ord[next++];
        if (rem[p] > 0) {
            q[qt++] = p;
        } else {
            ct[p] = t;
            done++;
        }
    }

    gantt(gp, gs, ge, g);
    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tNext CPU burst\n");
    for (i = 0; i < n; i++) {
        tat = ct[i] - at[i];
        wt = tat - bt[i];
        ttat += tat;
        twt += wt;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt, rand() % 10 + 1);
    }
    printf("\nAverage Turnaround Time = %.2f\n", ttat / n);
    printf("Average Waiting Time    = %.2f\n", twt / n);
    return 0;
}
```

Sample Input / Output

```
Enter number of processes: 4
Enter arrival time and burst time for P1: 0 5
Enter arrival time and burst time for P2: 1 3
Enter arrival time and burst time for P3: 2 1
Enter arrival time and burst time for P4: 3 2
Enter time quantum: 2

Gantt Chart:
|  P1  |  P2  |  P3  |  P1  |  P4  |  P2  |  P1  |
0      2      4      5      7      9      10     11

Process AT      BT      CT      TAT     WT      Next CPU burst
P1      0       5       11      11      6       1
P2      1       3       10      9       6       1
P3      2       1       5       3       2       8
P4      3       2       9       6       4       9

Average Turnaround Time = 7.25
Average Waiting Time    = 4.50
```

The question gives no data, so sample data is used. The Next CPU burst column is generated randomly with rand(), as the question asks, so it changes on every run.

Q2 (15 marks)

Question: Write a simulation program for disk scheduling using the LOOK algorithm. Requests: 55, 58, 39, 18, 90, 160, 150, 38. Start head position: 100, direction Left.

Program

```
#include <stdio.h>
#include <stdlib.h>

int cur, total = 0;

void go(int x) {
    printf(" -> %d", x);
    total += abs(x - cur);
    cur = x;
}

int main() {
    int req[50], n, blocks, head, dir, i, j, t, pos;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &blocks);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &req[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);
```

```
    printf("Enter direction (0 = Left, 1 = Right): ");
    scanf("%d", &dir);

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (req[j] > req[j + 1]) {
                t = req[j];
                req[j] = req[j + 1];
                req[j + 1] = t;
            }

    for (pos = 0; pos < n && req[pos] < head; pos++);

    cur = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0) {
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
        for (i = pos; i < n; i++)
            go(req[i]);
    } else {
        for (i = pos; i < n; i++)
            go(req[i]);
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
    }
    printf("\nTotal head movements = %d\n", total);
    return 0;
}
```

Sample Input / Output

```
Enter total number of disk blocks: 200
Enter number of requests: 8
Enter request string: 55 58 39 18 90 160 150 38
Enter current head position: 100
Enter direction (0 = Left, 1 = Right): 0

Order of service: 100 -> 90 -> 58 -> 55 -> 39 -> 38 -> 18 -> 150 -> 160
Total head movements = 224
```

SLIP 11

Q1 (15 marks)

Question: Write a C program to simulate Banker's algorithm for deadlock avoidance (5 processes, resource types A B C, Allocation, Max and Available as given in the question). Implement the menu: a. Accept Available b. Display Allocation, Max c. Display the contents of need matrix d. Display Available.

Program

```
#include <stdio.h>

int alloc[10][10], max[10][10], need[10][10], avail[10];
int n, m;

int main() {
    char ch;
    int i, j;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    printf("Enter Allocation matrix (%d x %d):\n", n, m);
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter Max matrix (%d x %d):\n", n, m);
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    do {
        printf("\na) Accept Available\n");
        printf("b) Display Allocation, Max\n");
        printf("c) Find Need and display it\n");
        printf("d) Display Available\n");
        printf("e) Exit\n");
        printf("Enter your choice: ");
        scanf(" %c", &ch);

        switch (ch) {
        case 'a':
            printf("Enter Available resources: ");
            for (j = 0; j < m; j++)
                scanf("%d", &avail[j]);
            break;
        case 'b':
            printf("\nProcess\tAllocation\tMax\n\t");
            for (j = 0; j < m; j++)
                printf("%c ", 'A' + j);
            printf("\t\t");
            for (j = 0; j < m; j++)
                printf("%c ", 'A' + j);
            printf("\n");
            for (i = 0; i < n; i++) {
                printf("P%d\t", i);
                for (j = 0; j < m; j++)
                    printf("%d ", alloc[i][j]);
                printf("\t\t");
                for (j = 0; j < m; j++)
                    printf("%d ", max[i][j]);
                printf("\n");
            }
            break;
        case 'c':
            printf("\nNeed Matrix (Max - Allocation)\n\t");
            for (j = 0; j < m; j++)
                printf("%c ", 'A' + j);
            printf("\n");
            for (i = 0; i < n; i++) {
                printf("P%d\t", i);
                for (j = 0; j < m; j++) {
                    need[i][j] = max[i][j] - alloc[i][j];
                    printf("%d ", need[i][j]);
```

```
                }
                printf("\n");
            }
            break;
        case 'd':
            printf("Available: ");
            for (j = 0; j < m; j++)
                printf("%c=%d ", 'A' + j, avail[j]);
            printf("\n");
            break;
        case 'e':
            printf("Exiting...\n");
            break;
        default:
            printf("Invalid choice\n");
        }
    } while (ch != 'e');
    return 0;
}
```

Sample Input / Output

```
Enter number of processes: 5
Enter number of resource types: 3
Enter Allocation matrix (5 x 3):
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
Enter Max matrix (5 x 3):
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2

a) Accept Available
b) Display Allocation, Max
c) Find Need and display it
d) Display Available
e) Exit
Enter your choice: a
Enter Available resources: 0 0 0

a) Accept Available
b) Display Allocation, Max
c) Find Need and display it
d) Display Available
e) Exit
Enter your choice: b

Process Allocation      Max
        A B C           A B C
P0      0 1 0           0 0 0
P1      2 0 0           2 0 2
P2      3 0 3           0 0 0
P3      2 1 1           1 0 0
P4      0 0 2           0 0 2

a) Accept Available
b) Display Allocation, Max
c) Find Need and display it
d) Display Available
e) Exit
Enter your choice: c

Need Matrix (Max - Allocation)
        A B C
P0      0 -1 0
P1      0 0 2
P2      -3 0 -3
P3      -1 -1 -1
P4      0 0 0

a) Accept Available
b) Display Allocation, Max
c) Find Need and display it
d) Display Available
e) Exit
```

```
Enter your choice: d
Available: A=0 B=0 C=0

a) Accept Available
b) Display Allocation, Max
c) Find Need and display it
d) Display Available
e) Exit
Enter your choice: e
Exiting...
```

Note: in the question's data some Max values are smaller than Allocation, so those need values come out negative. That is how the given data works out.

Q2 (15 marks)

Question: Write the simulation program for demand paging and show the page scheduling and total number of page faults according to the Optimal page replacement algorithm. Assume the memory of n frames. Reference string: 8,5,7,8,5,7,2,3,7,3,5,9,4,6,2. (Same as Slip 9 Q1)

Program

```c
#include <stdio.h>

int main() {
    int ref[50], frames[10], n, f, i, j, k, faults = 0, hit, filled = 0;
    int farthest, victim;

    printf("Enter number of frames: ");
    scanf("%d", &f);
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    for (i = 0; i < f; i++)
        frames[i] = -1;

    printf("\nRef\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        hit = 0;
        for (j = 0; j < f; j++)
            if (frames[j] == ref[i]) {
                hit = 1;
                break;
            }
        if (!hit) {
            faults++;
            if (filled < f) {
                frames[filled++] = ref[i];
            } else {
                farthest = -1;
                victim = 0;
                for (j = 0; j < f; j++) {
                    for (k = i + 1; k < n; k++)
                        if (frames[j] == ref[k])
                            break;
                    if (k > farthest) {
                        farthest = k;
                        victim = j;
                    }
                }
                frames[victim] = ref[i];
            }
        }
        printf("%d\t", ref[i]);
        for (j = 0; j < f; j++) {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
        printf("\t\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nTotal page faults = %d\n", faults);
    return 0;
```

```
}
```

Sample Input / Output

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 8 5 7 8 5 7 2 3 7 3 5 9 4 6 2

Ref     Frames          Status
8       8 - -           Fault
5       8 5 -           Fault
7       8 5 7           Fault
8       8 5 7           Hit
5       8 5 7           Hit
7       8 5 7           Hit
2       2 5 7           Fault
3       3 5 7           Fault
7       3 5 7           Hit
3       3 5 7           Hit
5       3 5 7           Hit
9       9 5 7           Fault
4       4 5 7           Fault
6       6 5 7           Fault
2       2 5 7           Fault

Total page faults = 9
```

SLIP 12

Q1 (15 marks)

Question: Write the program to simulate FCFS CPU scheduling. The arrival time and first CPU burst for n processes are input, the next CPU burst is generated randomly. Output the Gantt chart, turnaround time and waiting time of each process and the average waiting and turnaround time.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void gantt(int gp[], int gs[], int ge[], int g) {
    int i;
    printf("\nGantt Chart:\n");
    for (i = 0; i < g; i++) {
        if (gp[i] == 0)
            printf("| Idle ");
        else
            printf("|  P%d  ", gp[i]);
    }
    printf("|\n");
    for (i = 0; i < g; i++)
        printf("%-7d", gs[i]);
    printf("%d\n", ge[g - 1]);
}

int main() {
    int n, i, j, k, t = 0, tat, wt, tmp;
    int at[20], bt[20], ct[20], ord[20];
    int gp[40], gs[40], ge[40], g = 0;
    float twt = 0, ttat = 0;

    srand(time(NULL));
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter arrival time and burst time for P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        ord[i] = i;
    }

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (at[ord[j]] > at[ord[j + 1]]) {
                tmp = ord[j];
                ord[j] = ord[j + 1];
                ord[j + 1] = tmp;
            }

    for (k = 0; k < n; k++) {
        i = ord[k];
        if (t < at[i]) {
            gp[g] = 0;
            gs[g] = t;
            ge[g] = at[i];
            g++;
            t = at[i];
        }
        gp[g] = i + 1;
        gs[g] = t;
        t += bt[i];
        ge[g] = t;
        g++;
        ct[i] = t;
    }

    gantt(gp, gs, ge, g);
    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tNext CPU burst\n");
    for (i = 0; i < n; i++) {
        tat = ct[i] - at[i];
        wt = tat - bt[i];
        ttat += tat;
```

```
        twt += wt;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt, rand() % 10 + 1);
    }
    printf("\nAverage Turnaround Time = %.2f\n", ttat / n);
    printf("Average Waiting Time    = %.2f\n", twt / n);
    return 0;
}
```

Sample Input / Output

```
Enter number of processes: 4
Enter arrival time and burst time for P1: 0 5
Enter arrival time and burst time for P2: 1 3
Enter arrival time and burst time for P3: 2 8
Enter arrival time and burst time for P4: 3 6

Gantt Chart:
|  P1  |  P2  |  P3  |  P4  |
0      5      8      16     22

Process AT      BT      CT      TAT     WT      Next CPU burst
P1      0       5       5       5       0       8
P2      1       3       8       7       4       8
P3      2       8       16      14      6       8
P4      3       6       22      19      13      9

Average Turnaround Time = 11.25
Average Waiting Time    = 5.75
```

The question gives no data, so sample data is used. The Next CPU burst column is generated randomly with rand(), as the question asks, so it changes on every run.

Q2 (15 marks)

Question: Implement a C program to accept n integers to be sorted. Main function creates a child process using fork. Parent sorts the integers using bubble sort and waits for the child using wait; child sorts the integers using insertion sort.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void show(int a[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int main() {
    int a[50], b[50], n, i, j, t, key, pid;

    printf("Enter number of integers: ");
    scanf("%d", &n);
    printf("Enter %d integers: ", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        b[i] = a[i];
    }
    fflush(stdout);

    pid = fork();
    if (pid < 0) {
        printf("Fork failed\n");
        exit(1);
    } else if (pid == 0) {
        for (i = 1; i < n; i++) {
            key = b[i];
            j = i - 1;
            while (j >= 0 && b[j] > key) {
                b[j + 1] = b[j];
                j--;
            }
            b[j + 1] = key;
        }
        printf("Child  (insertion sort): ");
```

```
        show(b, n);
    } else {
        for (i = 0; i < n - 1; i++)
            for (j = 0; j < n - i - 1; j++)
                if (a[j] > a[j + 1]) {
                    t = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = t;
                }
        printf("Parent (bubble sort)   : ");
        show(a, n);
        wait(NULL);
        printf("Child process completed.\n");
    }
    return 0;
}
```

Sample Input / Output

```
Enter number of integers: 5
Enter 5 integers: 64 25 12 22 11
Parent (bubble sort)   : 11 12 22 25 64
Child  (insertion sort): 11 12 22 25 64
Child process completed.
```

SLIP 13

Q1 (15 marks)

Question: Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU burst for n processes are input, the next CPU burst is generated randomly. Output the Gantt chart, turnaround time and waiting time of each process and the average waiting and turnaround time. (Same as Slip 9 Q2)

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void gantt(int gp[], int gs[], int ge[], int g) {
    int i;
    printf("\nGantt Chart:\n");
    for (i = 0; i < g; i++) {
        if (gp[i] == 0)
            printf("| Idle ");
        else
            printf("|  P%d  ", gp[i]);
    }
    printf("|\n");
    for (i = 0; i < g; i++)
        printf("%-7d", gs[i]);
    printf("%d\n", ge[g - 1]);
}

int main() {
    int n, i, t = 0, done = 0, sel, next, tat, wt;
    int at[20], bt[20], ct[20], fin[20] = {0};
    int gp[40], gs[40], ge[40], g = 0;
    float twt = 0, ttat = 0;

    srand(time(NULL));
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter arrival time and burst time for P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    while (done < n) {
        sel = -1;
        for (i = 0; i < n; i++)
            if (!fin[i] && at[i] <= t && (sel == -1 || bt[i] < bt[sel]))
                sel = i;
        if (sel == -1) {
            next = -1;
            for (i = 0; i < n; i++)
                if (!fin[i] && (next == -1 || at[i] < at[next]))
                    next = i;
            gp[g] = 0;
            gs[g] = t;
            ge[g] = at[next];
            g++;
            t = at[next];
            continue;
        }
        gp[g] = sel + 1;
        gs[g] = t;
        t += bt[sel];
        ge[g] = t;
        g++;
        ct[sel] = t;
        fin[sel] = 1;
        done++;
    }

    gantt(gp, gs, ge, g);
    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tNext CPU burst\n");
    for (i = 0; i < n; i++) {
        tat = ct[i] - at[i];
        wt = tat - bt[i];
```

```
        ttat += tat;
        twt += wt;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt, rand() % 10 + 1);
    }
    printf("\nAverage Turnaround Time = %.2f\n", ttat / n);
    printf("Average Waiting Time    = %.2f\n", twt / n);
    return 0;
}
```

Sample Input / Output

```
Enter number of processes: 4
Enter arrival time and burst time for P1: 0 8
Enter arrival time and burst time for P2: 1 4
Enter arrival time and burst time for P3: 2 9
Enter arrival time and burst time for P4: 3 5

Gantt Chart:
|  P1  |  P2  |  P4  |  P3  |
0      8      12     17     26

Process AT      BT      CT      TAT     WT      Next CPU burst
P1      0       8       8       8       0       9
P2      1       4       12      11      7       2
P3      2       9       26      24      15      7
P4      3       5       17      14      9       7

Average Turnaround Time = 14.25
Average Waiting Time    = 7.75
```

The question gives no data, so sample data is used. The Next CPU burst column is generated randomly with rand(), as the question asks, so it changes on every run.

Q2 (15 marks)

Question: Write a simulation program for disk scheduling using the LOOK algorithm. Requests: 15, 30, 55, 90, 125, 140, 170. Starting head position: 100, direction Left. (Same program as Slip 10 Q2)

Program

```
#include <stdio.h>
#include <stdlib.h>

int cur, total = 0;

void go(int x) {
    printf(" -> %d", x);
    total += abs(x - cur);
    cur = x;
}

int main() {
    int req[50], n, blocks, head, dir, i, j, t, pos;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &blocks);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &req[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): ");
    scanf("%d", &dir);

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (req[j] > req[j + 1]) {
                t = req[j];
                req[j] = req[j + 1];
                req[j + 1] = t;
            }

    for (pos = 0; pos < n && req[pos] < head; pos++);

    cur = head;
    printf("\nOrder of service: %d", head);
```

```
    if (dir == 0) {
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
        for (i = pos; i < n; i++)
            go(req[i]);
    } else {
        for (i = pos; i < n; i++)
            go(req[i]);
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
    }
    printf("\nTotal head movements = %d\n", total);
    return 0;
}
```

Sample Input / Output

```
Enter total number of disk blocks: 200
Enter number of requests: 7
Enter request string: 15 30 55 90 125 140 170
Enter current head position: 100
Enter direction (0 = Left, 1 = Right): 0

Order of service: 100 -> 90 -> 55 -> 30 -> 15 -> 125 -> 140 -> 170
Total head movements = 240
```

SLIP 16

Q1 (15 marks)

Question: Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (counter method) page replacement algorithm. Assume the memory of n frames. Reference string: 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2.

Program

```
#include <stdio.h>

int main() {
    int ref[50], frames[10], last[10], n, f, i, j, faults = 0, hit;
    int filled = 0, victim;

    printf("Enter number of frames: ");
    scanf("%d", &f);
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    for (i = 0; i < f; i++) {
        frames[i] = -1;
        last[i] = 0;
    }

    printf("\nRef\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        hit = 0;
        for (j = 0; j < f; j++)
            if (frames[j] == ref[i]) {
                hit = 1;
                last[j] = i + 1;
                break;
            }
        if (!hit) {
            faults++;
            if (filled < f) {
                victim = filled++;
            } else {
                victim = 0;
                for (j = 1; j < f; j++)
                    if (last[j] < last[victim])
                        victim = j;
            }
            frames[victim] = ref[i];
            last[victim] = i + 1;
        }
        printf("%d\t", ref[i]);
        for (j = 0; j < f; j++) {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
        printf("\t\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nTotal page faults = %d\n", faults);
    return 0;
}
```

Sample Input / Output

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 3 5 7 2 5 1 2 3 1 3 5 3 1 6 2

Ref     Frames          Status
3       3 - -           Fault
5       3 5 -           Fault
7       3 5 7           Fault
2       2 5 7           Fault
5       2 5 7           Hit
```

```
1       2 5 1           Fault
2       2 5 1           Hit
3       2 3 1           Fault
1       2 3 1           Hit
3       2 3 1           Hit
5       5 3 1           Fault
3       5 3 1           Hit
1       5 3 1           Hit
6       6 3 1           Fault
2       6 2 1           Fault

Total page faults = 9
```

Q2 (15 marks)

Question: Write a C program that behaves like a shell which displays a command prompt. It accepts the command, tokenizes the command line and executes it by creating a child process. Also implement the additional command count: count c filename (number of characters), count w filename (number of words), count l filename (number of lines).

Program

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

void make_toks(char *s, char *tok[])
{
 int i = 0;
 char *p;
 p = strtok(s, " ");
 while (p != NULL)
 {
  tok[i++] = p;
  p = strtok(NULL, " ");
 }
 tok[i] = NULL;
}

void count(char *fn, char op)
{
 int fh, cc = 0, wc = 0, lc = 0;
 char c;
 fh = open(fn, O_RDONLY);
 if (fh == -1)
 {
  printf("File %s not found.\n", fn);
  return;
 }
 while (read(fh, &c, 1) > 0)
 {
  if (c == ' ')
   wc++;
  else if (c == '\n')
  {
   wc++;
   lc++;
  }
  cc++;
 }
 close(fh);
 switch (op)
 {
 case 'c':
  printf("No.of characters:%d\n", cc);
  break;
 case 'w':
  printf("No.of words:%d\n", wc);
  break;
 case 'l':
  printf("No.of lines:%d\n", lc);
  break;
 }
```

```
}

int main()
{
 char buff[80], *args[10];
 int pid;
 while (1)
 {
  printf("myshell$ ");
  fflush(stdout);
  fgets(buff, 80, stdin);
  buff[strlen(buff) - 1] = '\0';
  make_toks(buff, args);
  if (args[0] == NULL)
   continue;
  if (strcmp(args[0], "quit") == 0)
   break;
  else if (strcmp(args[0], "count") == 0)
  {
   if (args[1] == NULL || args[2] == NULL)
    printf("Usage: count c|w|l filename\n");
   else
    count(args[2], args[1][0]);
  }
  else
  {
   pid = fork();
   if (pid > 0)
    wait(NULL);
   else
   {
    if (execvp(args[0], args) == -1)
     printf("Bad command\n");
    exit(1);
   }
  }
 }
 return 0;
}
```

Sample Input / Output

```
myshell$ count c test.txt
No.of characters:39
myshell$ count w test.txt
No.of words:8
myshell$ count l test.txt
No.of lines:3
myshell$ ls test.txt
test.txt
myshell$ xyz
Bad command
myshell$ quit
```

Shell style follows your sample (myshell$ prompt, make_toks, count, fork/wait/execvp). Create a small test.txt before running. Words are counted by spaces and newlines, as in the sample.

SLIP 20

Q1 (15 marks)

Question: Write a simulation program for disk scheduling using the LOOK algorithm. Requests: 86, 147, 91, 177, 45, 12, 130. Starting head position: 60, direction Right. (Same program as Slip 10 Q2)

Program

```
#include <stdio.h>
#include <stdlib.h>

int cur, total = 0;

void go(int x) {
    printf(" -> %d", x);
    total += abs(x - cur);
    cur = x;
}

int main() {
    int req[50], n, blocks, head, dir, i, j, t, pos;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &blocks);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &req[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): ");
    scanf("%d", &dir);

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (req[j] > req[j + 1]) {
                t = req[j];
                req[j] = req[j + 1];
                req[j + 1] = t;
            }

    for (pos = 0; pos < n && req[pos] < head; pos++);

    cur = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0) {
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
        for (i = pos; i < n; i++)
            go(req[i]);
    } else {
        for (i = pos; i < n; i++)
            go(req[i]);
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
    }
    printf("\nTotal head movements = %d\n", total);
    return 0;
}
```

Sample Input / Output

```
Enter total number of disk blocks: 200
Enter number of requests: 7
Enter request string: 86 147 91 177 45 12 130
Enter current head position: 60
Enter direction (0 = Left, 1 = Right): 1

Order of service: 60 -> 86 -> 91 -> 130 -> 147 -> 177 -> 45 -> 12
Total head movements = 282
```

Q2 (15 marks)

Question: Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (counter method) page replacement algorithm. Assume the memory of n frames. Reference string: 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2. (Same as Slip 16 Q1)

Program

```
#include <stdio.h>

int main() {
    int ref[50], frames[10], last[10], n, f, i, j, faults = 0, hit;
    int filled = 0, victim;

    printf("Enter number of frames: ");
    scanf("%d", &f);
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    for (i = 0; i < f; i++) {
        frames[i] = -1;
        last[i] = 0;
    }

    printf("\nRef\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        hit = 0;
        for (j = 0; j < f; j++)
            if (frames[j] == ref[i]) {
                hit = 1;
                last[j] = i + 1;
                break;
            }
        if (!hit) {
            faults++;
            if (filled < f) {
                victim = filled++;
            } else {
                victim = 0;
                for (j = 1; j < f; j++)
                    if (last[j] < last[victim])
                        victim = j;
            }
            frames[victim] = ref[i];
            last[victim] = i + 1;
        }
        printf("%d\t", ref[i]);
        for (j = 0; j < f; j++) {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
        printf("\t\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nTotal page faults = %d\n", faults);
    return 0;
}
```

Sample Input / Output

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 3 5 7 2 5 1 2 3 1 3 5 3 1 6 2

Ref     Frames          Status
3       3 - -           Fault
5       3 5 -           Fault
7       3 5 7           Fault
2       2 5 7           Fault
5       2 5 7           Hit
1       2 5 1           Fault
2       2 5 1           Hit
3       2 3 1           Fault
1       2 3 1           Hit
```

```
3       2 3 1           Hit
5       5 3 1           Fault
3       5 3 1           Hit
1       5 3 1           Hit
6       6 3 1           Fault
2       6 2 1           Fault

Total page faults = 9
```

SLIP 23

Q1 (15 marks)

Question: Implement a C program to accept n integers to be sorted. Main function creates a child process using fork. Parent sorts the integers using bubble sort and waits for the child using wait; child sorts the integers using insertion sort. (Same as Slip 12 Q2)

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void show(int a[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int main() {
    int a[50], b[50], n, i, j, t, key, pid;

    printf("Enter number of integers: ");
    scanf("%d", &n);
    printf("Enter %d integers: ", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        b[i] = a[i];
    }
    fflush(stdout);

    pid = fork();
    if (pid < 0) {
        printf("Fork failed\n");
        exit(1);
    } else if (pid == 0) {
        for (i = 1; i < n; i++) {
            key = b[i];
            j = i - 1;
            while (j >= 0 && b[j] > key) {
                b[j + 1] = b[j];
                j--;
            }
            b[j + 1] = key;
        }
        printf("Child  (insertion sort): ");
        show(b, n);
    } else {
        for (i = 0; i < n - 1; i++)
            for (j = 0; j < n - i - 1; j++)
                if (a[j] > a[j + 1]) {
                    t = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = t;
                }
        printf("Parent (bubble sort)   : ");
        show(a, n);
        wait(NULL);
        printf("Child process completed.\n");
    }
    return 0;
}
```

Sample Input / Output

```
Enter number of integers: 6
Enter 6 integers: 50 30 10 40 20 60
Parent (bubble sort)   : 10 20 30 40 50 60
Child  (insertion sort): 10 20 30 40 50 60
Child process completed.
```

Q2 (15 marks)

Question: Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU burst for n processes are input, the next CPU burst is generated randomly. Output the Gantt chart, turnaround time and waiting time of each process and the average waiting and turnaround time. (Same as Slip 9 Q2)

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void gantt(int gp[], int gs[], int ge[], int g) {
    int i;
    printf("\nGantt Chart:\n");
    for (i = 0; i < g; i++) {
        if (gp[i] == 0)
            printf("| Idle ");
        else
            printf("|  P%d  ", gp[i]);
    }
    printf("|\n");
    for (i = 0; i < g; i++)
        printf("%-7d", gs[i]);
    printf("%d\n", ge[g - 1]);
}

int main() {
    int n, i, t = 0, done = 0, sel, next, tat, wt;
    int at[20], bt[20], ct[20], fin[20] = {0};
    int gp[40], gs[40], ge[40], g = 0;
    float twt = 0, ttat = 0;

    srand(time(NULL));
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter arrival time and burst time for P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    while (done < n) {
        sel = -1;
        for (i = 0; i < n; i++)
            if (!fin[i] && at[i] <= t && (sel == -1 || bt[i] < bt[sel]))
                sel = i;
        if (sel == -1) {
            next = -1;
            for (i = 0; i < n; i++)
                if (!fin[i] && (next == -1 || at[i] < at[next]))
                    next = i;
            gp[g] = 0;
            gs[g] = t;
            ge[g] = at[next];
            g++;
            t = at[next];
            continue;
        }
        gp[g] = sel + 1;
        gs[g] = t;
        t += bt[sel];
        ge[g] = t;
        g++;
        ct[sel] = t;
        fin[sel] = 1;
        done++;
    }

    gantt(gp, gs, ge, g);
    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tNext CPU burst\n");
    for (i = 0; i < n; i++) {
        tat = ct[i] - at[i];
        wt = tat - bt[i];
        ttat += tat;
        twt += wt;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt, rand() % 10 + 1);
```

```
    }
    printf("\nAverage Turnaround Time = %.2f\n", ttat / n);
    printf("Average Waiting Time    = %.2f\n", twt / n);
    return 0;
}
```

Sample Input / Output

```
Enter number of processes: 4
Enter arrival time and burst time for P1: 1 6
Enter arrival time and burst time for P2: 2 8
Enter arrival time and burst time for P3: 3 7
Enter arrival time and burst time for P4: 4 3

Gantt Chart:
| Idle |  P1  |  P4  |  P3  |  P2  |
0      1      7      10     17     25

Process AT      BT      CT      TAT     WT      Next CPU burst
P1      1       6       7       6       0       6
P2      2       8       25      23      15      8
P3      3       7       17      14      7       10
P4      4       3       10      6       3       5

Average Turnaround Time = 12.25
Average Waiting Time    = 6.25
```

The question gives no data, so sample data is used. The Next CPU burst column is generated randomly with rand(), as the question asks, so it changes on every run.

SLIP 25

Q1 (15 marks)

Question: Write a C program to illustrate the concept of an orphan process. Parent creates a child and terminates before the child has finished its task, so the child becomes an orphan. (Use fork(), sleep(), getpid(), getppid())

Program

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }
    if (pid == 0) {
        printf("Child : PID = %d, Parent PID = %d\n", getpid(), getppid());
        sleep(5);
        printf("Child (now orphan): PID = %d, New Parent PID = %d\n", getpid(), getppid());
    } else {
        printf("Parent: PID = %d, Child PID = %d\n", getpid(), pid);
        printf("Parent terminating before child finishes.\n");
    }
    return 0;
}
```

Sample Input / Output

```
$ ./orphan
Parent: PID = 8120, Child PID = 8121
Child : PID = 8121, Parent PID = 8120
Parent terminating before child finishes.
$
(after about 5 seconds)
Child (now orphan): PID = 8121, New Parent PID = 1
```

Note: PIDs differ on every run. The last line appears about 5 seconds after the parent has ended; New Parent PID is usually 1 (init/systemd).

Q2 (15 marks)

Question: Write a C program that behaves like a shell which displays a command prompt. It accepts the command, tokenizes the command line and executes it by creating a child process. Also implement the additional command search: search f filename pattern (first occurrence), search a filename pattern (all occurrences), search c filename pattern (count of occurrences).

Program

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

void make_toks(char *s, char *tok[])
{
 int i = 0;
 char *p;
 p = strtok(s, " ");
 while (p != NULL)
 {
  tok[i++] = p;
  p = strtok(NULL, " ");
 }
 tok[i] = NULL;
}

void search(char op, char *fn, char *pat)
{
 FILE *fp;
```

```
 char line[200], *p;
 int ln = 0, cnt = 0;
 fp = fopen(fn, "r");
 if (fp == NULL)
 {
  printf("File %s not found.\n", fn);
  return;
 }
 while (fgets(line, 200, fp) != NULL)
 {
  ln++;
  p = line;
  while ((p = strstr(p, pat)) != NULL)
  {
   cnt++;
   if (op == 'f' || op == 'a')
    printf("Found at line %d, position %d: %s", ln, (int)(p - line) + 1, line);
   if (op == 'f')
   {
    fclose(fp);
    return;
   }
   p = p + strlen(pat);
  }
 }
 fclose(fp);
 if (op == 'c')
  printf("Pattern \"%s\" occurs %d time(s)\n", pat, cnt);
 else if (cnt == 0)
  printf("Pattern not found\n");
}

int main()
{
 char buff[80], *args[10];
 int pid;
 while (1)
 {
  printf("myshell$ ");
  fflush(stdout);
  fgets(buff, 80, stdin);
  buff[strlen(buff) - 1] = '\0';
  make_toks(buff, args);
  if (args[0] == NULL)
   continue;
  if (strcmp(args[0], "quit") == 0)
   break;
  else if (strcmp(args[0], "search") == 0)
  {
   if (args[1] == NULL || args[2] == NULL || args[3] == NULL)
    printf("Usage: search f|a|c filename pattern\n");
   else
    search(args[1][0], args[2], args[3]);
  }
  else
  {
   pid = fork();
   if (pid > 0)
    wait(NULL);
   else
   {
    if (execvp(args[0], args) == -1)
     printf("Bad command\n");
    exit(1);
   }
  }
 }
 return 0;
}
```

Sample Input / Output

```
myshell$ search f search.txt hello
Found at line 1, position 1: hello world
myshell$ search a search.txt hello
Found at line 1, position 1: hello world
Found at line 3, position 1: hello again
```

```
myshell$ search c search.txt is
Pattern "is" occurs 2 time(s)
myshell$ xyz
Bad command
myshell$ quit
```

Shell style follows your sample. Create search.txt (for example: hello world / this is a test / hello again) before running.

SLIP 27

Q1 (15 marks)

Question: Write a simulation program for disk scheduling using the SSTF algorithm. Requests: 30, 10, 60, 95, 120, 150, 175. Start head position: 50. (Same program as Slip 3 Q2)

Program

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, blocks, head, i, j, total = 0, served = 0;
    int req[50], done[50] = {0};

    printf("Enter total number of disk blocks: ");
    scanf("%d", &blocks);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &req[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);

    printf("\nOrder of service: %d", head);
    while (served < n) {
        int min = 100000, idx = -1;
        for (j = 0; j < n; j++) {
            if (!done[j] && abs(req[j] - head) < min) {
                min = abs(req[j] - head);
                idx = j;
            }
        }
        done[idx] = 1;
        total += min;
        head = req[idx];
        printf(" -> %d", head);
        served++;
    }
    printf("\nTotal head movements = %d\n", total);
    return 0;
}
```

Sample Input / Output

```
Enter total number of disk blocks: 200
Enter number of requests: 7
Enter request string: 30 10 60 95 120 150 175
Enter current head position: 50

Order of service: 50 -> 60 -> 30 -> 10 -> 95 -> 120 -> 150 -> 175
Total head movements = 225
```

Q2 (15 marks)

Question: Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (counter method) page replacement algorithm. Assume the memory of n frames. Reference string: 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2. (Same as Slip 16 Q1)

Program

```
#include <stdio.h>

int main() {
    int ref[50], frames[10], last[10], n, f, i, j, faults = 0, hit;
    int filled = 0, victim;

    printf("Enter number of frames: ");
    scanf("%d", &f);
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    for (i = 0; i < f; i++) {
        frames[i] = -1;
        last[i] = 0;
    }

    printf("\nRef\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        hit = 0;
        for (j = 0; j < f; j++)
            if (frames[j] == ref[i]) {
                hit = 1;
                last[j] = i + 1;
                break;
            }
        if (!hit) {
            faults++;
            if (filled < f) {
                victim = filled++;
            } else {
                victim = 0;
                for (j = 1; j < f; j++)
                    if (last[j] < last[victim])
                        victim = j;
            }
            frames[victim] = ref[i];
            last[victim] = i + 1;
        }
        printf("%d\t", ref[i]);
        for (j = 0; j < f; j++) {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
        printf("\t\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nTotal page faults = %d\n", faults);
    return 0;
}
```

Sample Input / Output

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 3 5 7 2 5 1 2 3 1 3 5 3 1 6 2

Ref     Frames          Status
3       3 - -           Fault
5       3 5 -           Fault
7       3 5 7           Fault
2       2 5 7           Fault
5       2 5 7           Hit
1       2 5 1           Fault
2       2 5 1           Hit
3       2 3 1           Fault
1       2 3 1           Hit
```

```
3       2 3 1           Hit
5       5 3 1           Fault
3       5 3 1           Hit
1       5 3 1           Hit
6       6 3 1           Fault
2       6 2 1           Fault

Total page faults = 9
```

SLIP 28

Q1 (15 marks)

Question: Write a simulation program for disk scheduling using the SCAN algorithm. Requests: 10, 25, 75, 90, 130, 145, 180, 55. Starting head position: 80, direction Left. (Same program as Slip 4 Q2)

Program

```
#include <stdio.h>
#include <stdlib.h>

int cur, total = 0;

void go(int x) {
    printf(" -> %d", x);
    total += abs(x - cur);
    cur = x;
}

int main() {
    int req[50], n, blocks, head, dir, i, j, t, pos;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &blocks);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &req[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): ");
    scanf("%d", &dir);

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (req[j] > req[j + 1]) {
                t = req[j];
                req[j] = req[j + 1];
                req[j + 1] = t;
            }

    for (pos = 0; pos < n && req[pos] < head; pos++);

    cur = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0) {
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
        go(0);
        for (i = pos; i < n; i++)
            go(req[i]);
    } else {
        for (i = pos; i < n; i++)
            go(req[i]);
        go(blocks - 1);
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
    }
    printf("\nTotal head movements = %d\n", total);
    return 0;
}
```

Sample Input / Output

```
Enter total number of disk blocks: 200
Enter number of requests: 8
Enter request string: 10 25 75 90 130 145 180 55
Enter current head position: 80
Enter direction (0 = Left, 1 = Right): 0

Order of service: 80 -> 75 -> 55 -> 25 -> 10 -> 0 -> 90 -> 130 -> 145 -> 180
Total head movements = 260
```

Q2 (15 marks)

Question: Write the simulation program for demand paging and show the page scheduling and total number of page faults according to the MFU page replacement algorithm. Assume the memory of n frames. Reference string: 8,5,7,8,5,7,2,3,7,3,5,9,4,6,2.

Program

```
#include <stdio.h>

int main() {
    int ref[50], frames[10], cnt[10], ld[10], n, f, i, j, faults = 0, hit;
    int filled = 0, victim;

    printf("Enter number of frames: ");
    scanf("%d", &f);
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    for (i = 0; i < f; i++) {
        frames[i] = -1;
        cnt[i] = 0;
        ld[i] = 0;
    }

    printf("\nRef\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        hit = 0;
        for (j = 0; j < f; j++)
            if (frames[j] == ref[i]) {
                hit = 1;
                cnt[j]++;
                break;
            }
        if (!hit) {
            faults++;
            if (filled < f) {
                victim = filled++;
            } else {
                victim = 0;
                for (j = 1; j < f; j++)
                    if (cnt[j] > cnt[victim] || (cnt[j] == cnt[victim] && ld[j] < ld[victim]))
                        victim = j;
            }
            frames[victim] = ref[i];
            cnt[victim] = 1;
            ld[victim] = i;
        }
        printf("%d\t", ref[i]);
        for (j = 0; j < f; j++) {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
        printf("\t\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nTotal page faults = %d\n", faults);
    return 0;
}
```

Sample Input / Output

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 8 5 7 8 5 7 2 3 7 3 5 9 4 6 2

Ref     Frames          Status
8       8 - -           Fault
5       8 5 -           Fault
7       8 5 7           Fault
8       8 5 7           Hit
5       8 5 7           Hit
7       8 5 7           Hit
```

```
2       2 5 7           Fault
3       2 3 7           Fault
7       2 3 7           Hit
3       2 3 7           Hit
5       2 3 5           Fault
9       2 9 5           Fault
4       4 9 5           Fault
6       4 9 6           Fault
2       4 2 6           Fault

Total page faults = 10
```

MFU replaces the page with the highest use count (ties go to the page loaded earliest).